COL 724 Assignment 2

Part C

Ankita Singh(2024MCS2891)

Vipul Nayyar(2024CSZ8495)

1. Part 1: Controller Hub and Learning Switch

The SDN controller implementations for this experiment have been done in **Python** using the **Ryu SDN framework**[1]. The Ryu framework provides a set of libraries to build OpenFlow controllers and handle switch events such as `packet_in` and `port_status`. Python's built-in `logging` module is used for event tracing, and the `ryu.lib.packet` library is used for packet parsing and protocol handling.

Two controller applications were implemented: **HubController** and **SimpleSwitch (Learning Switch)**.

- **HubController**: Implements a basic hub behavior by forwarding all incoming packets to all ports except the source port. It does not install flow entries on the switch.

- **SimpleSwitch (Learning Switch)**: Implements a learning switch behavior. It dynamically learns MAC-to-port mappings from incoming traffic and installs flow rules to forward packets directly to the correct port, flooding only for unknown destinations.

Network Topology:

The network topology is defined using **Mininet**[2] with 5 hosts (h1–h5) and 2 Open vSwitch switches (s1, s2). Hosts are connected to the switches as follows: h1–h3 to s1 and h4–h5 to s2, with an inter-switch link connecting s1 and s2. The Mininet topology and controller communication are configured to use **OpenFlow v1.0**.

Implementation:

The SDN controllers were implemented using **Ryu framework in Python** with **OpenFlow v1.0**, and the network topology was created in **Mininet** with 5 hosts and 2 switches.

Algorithm / Workflow:

Hub Controller

The **Hub Controller** operates like a traditional network hub. It listens for incoming packets from the switches and forwards each packet to all other ports except the incoming port. It does not install any flow entries, and every packet is processed by the controller, resulting in high network overhead and redundant traffic.

Learning Switch (SimpleSwitch)

The **Learning Switch** operates by dynamically learning MAC addresses from incoming packets and associating them with the corresponding switch ports. For each packet, if the destination MAC address is known, it forwards the packet directly to the correct port and installs a flow entry in the switch to handle future packets without involving the controller. If the destination is unknown, the packet is flooded to all other ports. Over time, this reduces flooding, optimizes network traffic, and lowers the controller load.

Experiments

Experiment 1: Connectivity Test (`pingall`)

We performed this experiment to verify that all hosts in the network could communicate with each other and to observe how the two controllers handle packet forwarding at the data link layer.

Procedure:

To test the experiment 1, started the Mininet topology with the **Hub Controller** and ran **`pingall`** to test connectivity between all hosts. Then, repeated the same test with the **Learning Switch** controller. We also checked the flow table entries on the switches using `dpctl dump-flows`.

Result:

```
(py_3.9.6) baadalvm@baadalvm:~/dev/COL724-Assignment-2/C$ sudo python3.10 p1_topo.py
*** Creating network
*** Adding hosts:
h1 h2 h3 h4 h5
*** Adding switches:
s1 s2
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s2) (h5, s2) (s1, s2)
*** Configuring hosts
h1 h2 h3 h4 h5
*** Starting controller
c0
*** Starting 2 switches
s1 s2 ...
*** Running CLI
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5
h2 -> h1 h3 h4 h5
h3 -> h1 h2 h4 h5
h4 -> h1 h2 h3 h5
h5 -> h1 h2 h3 h4
*** Results: 0% dropped (20/20 received)
mininet> dpctl dump-flows
*** s1 ------------------------------------------------------------------------
*** s2 ------------------------------------------------------------------------
mininet> iperf
*** Iperf: testing TCP bandwidth between h1 and h5
*** Results: ['26.0 Mbits/sec', '25.9 Mbits/sec']
mininet>
```

Figure 1.1: Screenshot of running p1_topo.py with Hub Controller

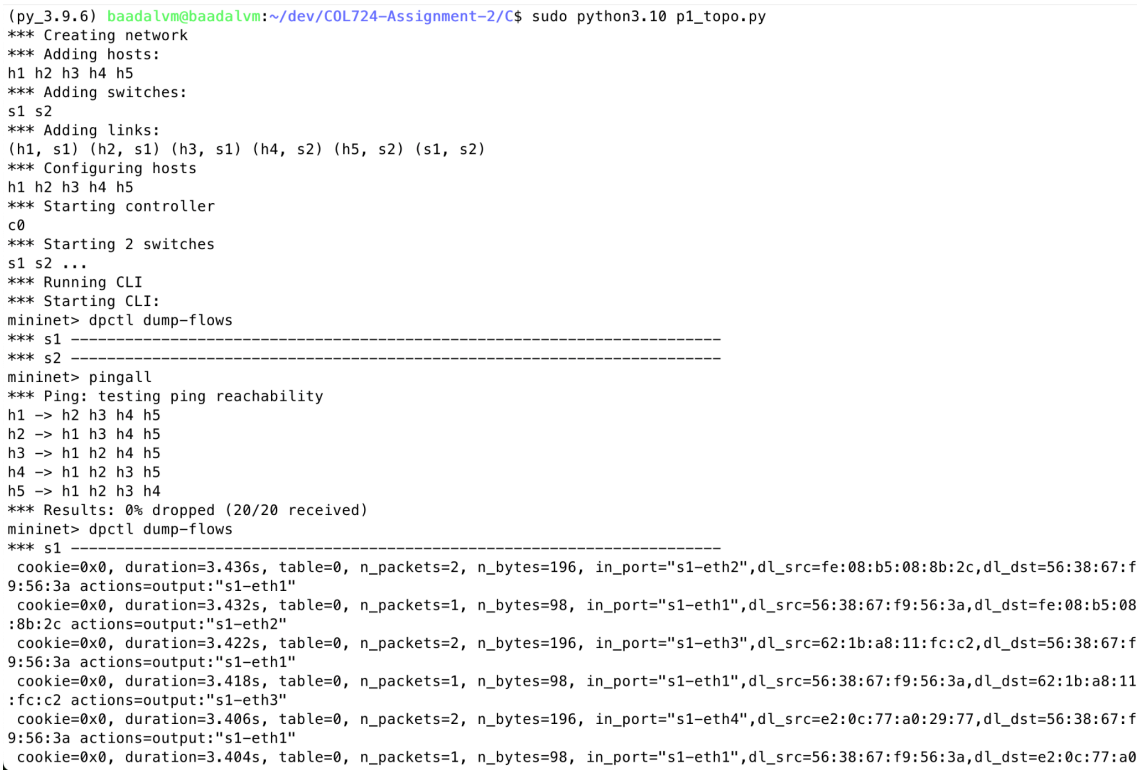


Figure 1.2: Screenshot of running p1_topo.py with Learning Switch(Part1)

Figure 1.1 shows the run log of Hub Controller and Figure 1.2 and 1.3 show the run log of the Learning Switch Controller

Observation: All hosts (h1–h5) were able to successfully communicate with each other.

Flow Table Rules: Both switches (s1 and s2) show **no flow entries installed**, as expected for a Hub Controller.

Explanation: Since the Hub Controller floods all incoming packets to every port, there is no need to install flows. This ensures connectivity, but every packet is sent to all ports, leading to redundant traffic and higher controller overhead.

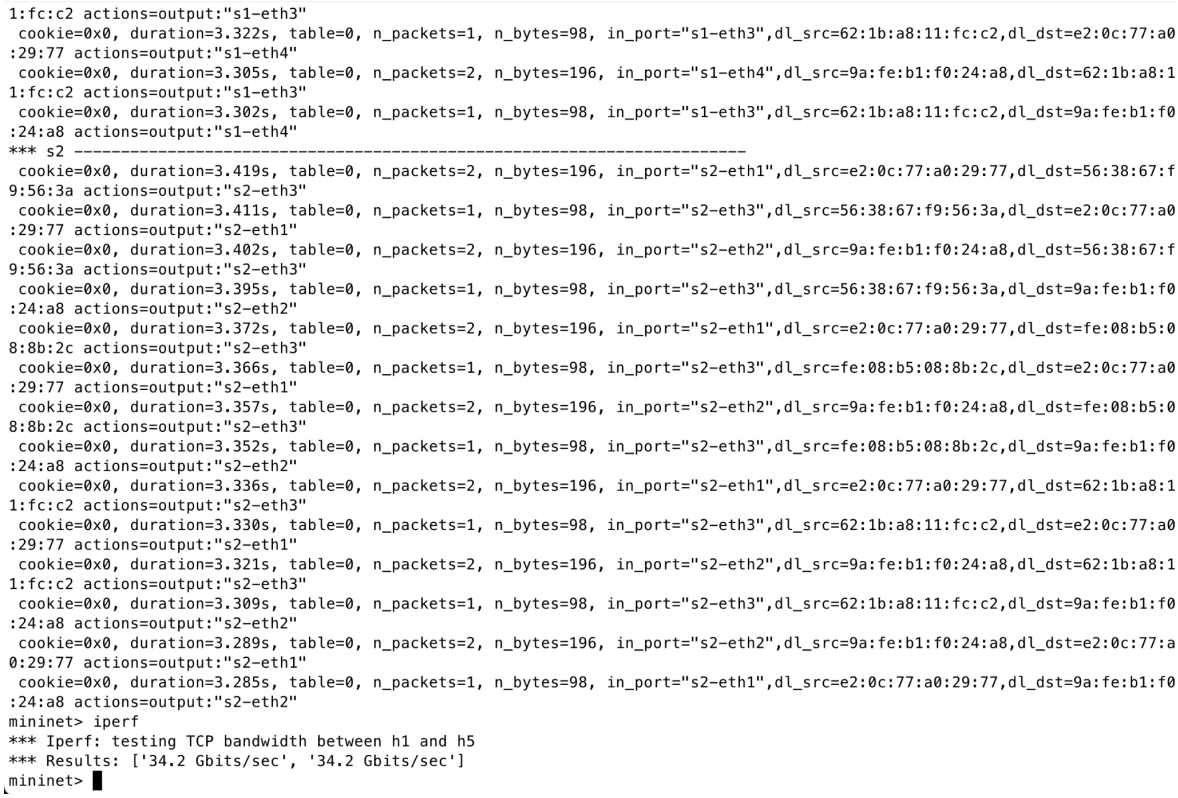


Figure 1.3: Screenshot of running p1_topo.py with Learning Switch(Part2)

Experiment 2: Throughput Test (`iperf`)

We conducted this experiment to compare the network performance of the two controllers under load and see how their forwarding strategies affect throughput between hosts.

Procedure:

For Experiment 2, measured TCP throughput between h1 and h5 using `iperf`. First, We ran the test with the Hub Controller, then repeated it with the Learning Switch in Figure 1.1 and Figure 1.3.

Observations:

The Hub controller in Figure 1.1 showed an iperf bandwidth of **26 Mbits/s** between Host h1 and h5. The Learning Switch controller showed an iperf bandwidth between Host h1 and h5 of **34.2 Gbits/s**. This is due to the learning behavior of the second controller, and adding those flow rules in the corresponding switch. This decreases the overhead of sending every packet to the controller.

Part 2: Shortest Path Routing(Using static weights for flows)

Each switch-to-switch link is configured with a bandwidth(Mbit) and delay(ms). The capacity of a switch-to-switch link is calculated using the Bandwidth delay product, which calculates how much data can be in transit on a link at a time. The inverse of this Bandwidth delay product acts as the weight for that particular link, while calculating the shortest path.

RTT = 2 * (delay(ms) / 1000)

Bandwidth delay product = RTT * bandwidth(Mbits)

Link Weight = 1 / (Bandwidth delay product)

1. Circular Topology

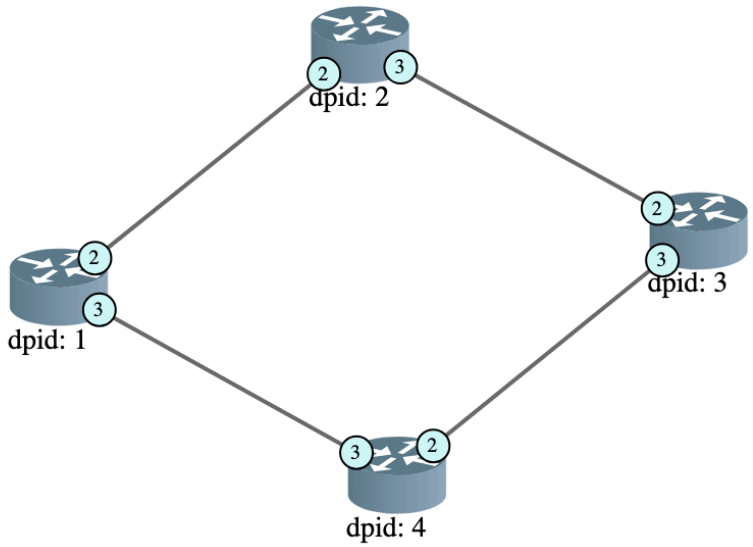


Figure 2.1: Switch 1 - 4 topology. Each Switch has one Host connected to it

```
Hosts:
Host 00:00:00:00:00:01
Host 00:00:00:00:00:02
Host 00:00:00:00:00:03
Host 00:00:00:00:00:04
Switches:
Switch 1
Switch 2
Switch 3
Switch 4
Links:
Link from switch 1 to its own port 1 : (1, '1a:a9:01:fb:21:91')  Delay: 0 Bw: 0 Weight: 0
Link from switch 1 to its own port 2 : (1, '8a:48:83:32:3c:cc')  Delay: 0 Bw: 0 Weight: 0
Link from switch 1 to its own port 3 : (1, '0e:46:9b:c2:7d:c8')  Delay: 0 Bw: 0 Weight: 0
Link from host to switch 1 : ('1a:a9:01:fb:21:91', '00:00:00:00:00:01')  Delay: 0 Bw: 0 Weight: 0
Link from switch 1 to switch 2 : ('8a:48:83:32:3c:cc', '22:38:d1:6e:84:75')  Delay: 0.01 Bw: 10.0 Weight: 5.0
Link from switch 1 to switch 4 : ('0e:46:9b:c2:7d:c8', '26:6e:13:b2:13:ed')  Delay: 0.01 Bw: 1.0 Weight: 50.0
Link from switch 2 to its own port 1 : (2, '86:2b:fc:7b:d6:b9')  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to its own port 2 : (2, '22:38:d1:6e:84:75')  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to its own port 3 : (2, '4e:51:52:7e:ea:f5')  Delay: 0 Bw: 0 Weight: 0
Link from host to switch 2 : ('86:2b:fc:7b:d6:b9', '00:00:00:00:00:02')  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to switch 3 : ('4e:51:52:7e:ea:f5', '8e:16:cf:0f:2e:79')  Delay: 0.01 Bw: 10.0 Weight: 5.0
Link from switch 3 to its own port 2 : ('8e:16:cf:0f:2e:79', 3)  Delay: 0 Bw: 0 Weight: 0
Link from switch 3 to its own port 1 : (3, '02:2e:fd:55:4f:ee')  Delay: 0 Bw: 0 Weight: 0
Link from switch 3 to its own port 3 : (3, 'c2:09:23:8d:0e:8e')  Delay: 0 Bw: 0 Weight: 0
Link from host to switch 3 : ('02:2e:fd:55:4f:ee', '00:00:00:00:00:03')  Delay: 0 Bw: 0 Weight: 0
Link from switch 4 to switch 3 : ('c2:09:23:8d:0e:8e', '76:8f:d2:e0:5d:be')  Delay: 0.01 Bw: 10.0 Weight: 5.0
Link from switch 4 to its own port 1 : (4, 'ae:e6:49:9b:7f:06')  Delay: 0 Bw: 0 Weight: 0
Link from switch 4 to its own port 2 : (4, '76:8f:d2:e0:5d:be')  Delay: 0 Bw: 0 Weight: 0
Link from switch 4 to its own port 3 : (4, '26:6e:13:b2:13:ed')  Delay: 0 Bw: 0 Weight: 0
Link from host to switch 4 : ('ae:e6:49:9b:7f:06', '00:00:00:00:00:04')  Delay: 0 Bw: 0 Weight: 0

**************************************
Shortest path from Host interface 00:00:00:00:00:01 to Host interface 00:00:00:00:00:02:

-> (Host Interface)00:00:00:00:00:01 -> (Switch Interface)1a:a9:01:fb:21:91 -> (Switch)1 -> (Switch Interface)8a:48:83:32:3c:cc -> (Switch Interface)
22:38:d1:6e:84:75 -> (Switch)2 -> (Switch Interface)86:2b:fc:7b:d6:b9 -> (Host Interface)00:00:00:00:00:02

**************************************
Shortest path from Host interface 00:00:00:00:00:01 to Host interface 00:00:00:00:00:03:

-> (Host Interface)00:00:00:00:00:01 -> (Switch Interface)1a:a9:01:fb:21:91 -> (Switch)1 -> (Switch Interface)8a:48:83:32:3c:cc -> (Switch Interface)
22:38:d1:6e:84:75 -> (Switch)2 -> (Switch Interface)4e:51:52:7e:ea:f5 -> (Switch Interface)8e:16:cf:0f:2e:79 -> (Switch)3 -> (Switch Interface)
02:2e:fd:55:4f:ee -> (Host Interface)00:00:00:00:00:03

**************************************
Shortest path from Host interface 00:00:00:00:00:01 to Host interface 00:00:00:00:00:04:

-> (Host Interface)00:00:00:00:00:01 -> (Switch Interface)1a:a9:01:fb:21:91 -> (Switch)1 -> (Switch Interface)8a:48:83:32:3c:cc -> (Switch Interface)
22:38:d1:6e:84:75 -> (Switch)2 -> (Switch Interface)4e:51:52:7e:ea:f5 -> (Switch Interface)8e:16:cf:0f:2e:79 -> (Switch)3 -> (Switch Interface)
c2:09:23:8d:0e:8e -> (Switch Interface)76:8f:d2:e0:5d:be -> (Switch)4 -> (Switch Interface)ae:e6:49:9b:7f:06 -> (Host Interface)00:00:00:00:00:04

**************************************
Shortest path from Host interface 00:00:00:00:00:02 to Host interface 00:00:00:00:00:03:

-> (Host Interface)00:00:00:00:00:02 -> (Switch Interface)86:2b:fc:7b:d6:b9 -> (Switch)2 -> (Switch Interface)4e:51:52:7e:ea:f5 -> (Switch Interface)
8e:16:cf:0f:2e:79 -> (Switch)3 -> (Switch Interface)02:2e:fd:55:4f:ee -> (Host Interface)00:00:00:00:00:03

**************************************
Shortest path from Host interface 00:00:00:00:00:02 to Host interface 00:00:00:00:00:04:

-> (Host Interface)00:00:00:00:00:02 -> (Switch Interface)86:2b:fc:7b:d6:b9 -> (Switch)2 -> (Switch Interface)4e:51:52:7e:ea:f5 -> (Switch Interface)
8e:16:cf:0f:2e:79 -> (Switch)3 -> (Switch Interface)c2:09:23:8d:0e:8e -> (Switch Interface)76:8f:d2:e0:5d:be -> (Switch)4 -> (Switch Interface)
ae:e6:49:9b:7f:06 -> (Host Interface)00:00:00:00:00:04

**************************************
Shortest path from Host interface 00:00:00:00:00:03 to Host interface 00:00:00:00:00:04:

-> (Host Interface)00:00:00:00:00:03 -> (Switch Interface)02:2e:fd:55:4f:ee -> (Switch)3 -> (Switch Interface)c2:09:23:8d:0e:8e -> (Switch Interface)
76:8f:d2:e0:5d:be -> (Switch)4 -> (Switch Interface)ae:e6:49:9b:7f:06 -> (Host Interface)00:00:00:00:00:04
```

Figure 2.2: Circular Topology data in log format(topology.txt)

Figure 2.1 and 2.2 show the Circular topology used to test Shortest Path Routing with static link weights. In Figure 2.2, Lines 2-5 show Hosts and their respective MAC addresses. Lines 7-10 show switches. Lines 12-31 show uni-directional links with their respective delay, bandwidth, and calculated link weights. Line 33 onwards lists the shortest path for all possible unique host pairs.

Note that in Figure 2.2, Switch 1 is directly connected to Switch 4(Line 17, weight is 50.0). The weights associated with links between Switch 1 to 2(Line 16), Switch 2 to 3(Line 22), and Switch 3 to 4(Line 27) are 5.0 each. This causes Switch 1 to have a path to Switch 4 going through Switch 2 and 3(Line 44 and 46), despite having a direct link since the weight of link between Switch 1 and 4 is too high.

2. Tree Topology

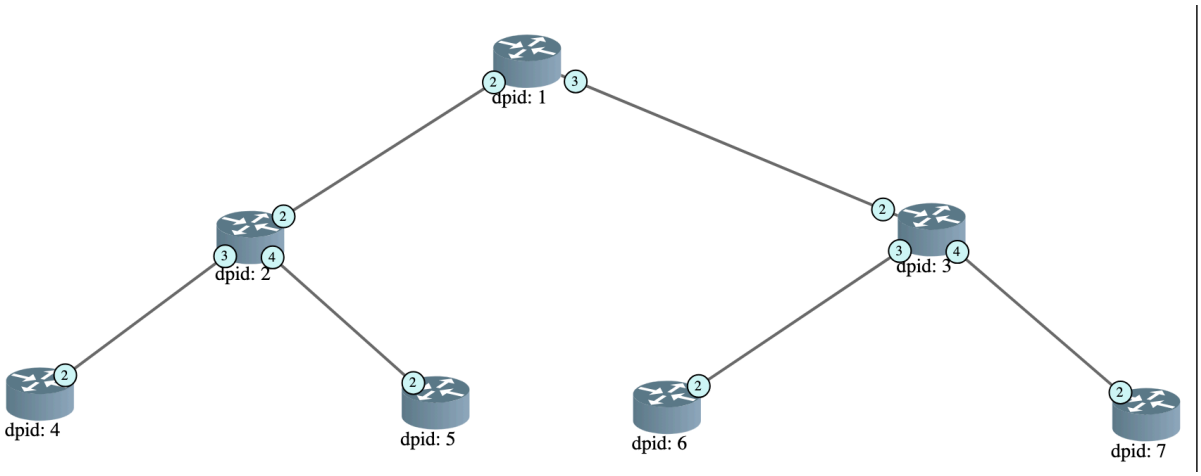


Figure 2.3: Tree Topology with 7 Switches, each connected to one Host each

```
Hosts:
Host 00:00:00:00:00:01
Host 00:00:00:00:00:02
Host 00:00:00:00:00:03
Host 00:00:00:00:00:04
Host 00:00:00:00:00:05
Host 00:00:00:00:00:06
Host 00:00:00:00:00:07
Switches:
Switch 1
Switch 2
Switch 3
Switch 4
Switch 5
Switch 6
Switch 7
Links:
Link from switch 1 to its own port 1 : (1, 'de:31:37:58:de:30')  Delay: 0 Bw: 0 Weight: 0
Link from switch 1 to its own port 2 : (1, '5e:b6:a6:e0:6f:46')  Delay: 0 Bw: 0 Weight: 0
Link from switch 1 to its own port 3 : (1, '9a:2c:01:ae:52:85')  Delay: 0 Bw: 0 Weight: 0
Link from host to switch 1 : ('de:31:37:58:de:30', '00:00:00:00:00:01')  Delay: 0 Bw: 0 Weight: 0
Link from switch 1 to switch 2 : ('5e:b6:a6:e0:6f:46', 'e6:13:7b:89:15:ae')  Delay: 0.01 Bw: 1.0 Weight: 50.
Link from switch 1 to switch 3 : ('9a:2c:01:ae:52:85', '5a:1b:70:10:d8:5d')  Delay: 0.01 Bw: 10.0 Weight: 5.
Link from switch 2 to its own port 1 : (2, 'b6:ff:12:eb:7b:cc')  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to its own port 4 : (2, '0a:19:d9:77:bf:05')  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to its own port 2 : (2, 'e6:13:7b:89:15:ae')  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to its own port 3 : (2, '9e:2f:c3:82:bd:15')  Delay: 0 Bw: 0 Weight: 0
Link from host to switch 2 : ('b6:ff:12:eb:7b:cc', '00:00:00:00:00:02')  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to switch 5 : ('0a:19:d9:77:bf:05', '6a:2b:49:84:4f:56')  Delay: 0.01 Bw: 10.0 Weight: 5.
Link from switch 2 to switch 4 : ('9e:2f:c3:82:bd:15', 'ee:f9:f6:81:0f:9e')  Delay: 0.01 Bw: 10.0 Weight: 5.
Link from switch 5 to its own port 2 : ('6a:2b:49:84:4f:56', 5)  Delay: 0 Bw: 0 Weight: 0
Link from switch 3 to its own port 2 : ('5a:1b:70:10:d8:5d', 3)  Delay: 0 Bw: 0 Weight: 0
Link from switch 3 to its own port 4 : (3, '96:ac:6f:e4:49:b4')  Delay: 0 Bw: 0 Weight: 0
Link from switch 3 to its own port 1 : (3, 'fa:c4:2c:ea:4f:dd')  Delay: 0 Bw: 0 Weight: 0
Link from switch 3 to its own port 3 : (3, '52:62:c2:56:4e:0f')  Delay: 0 Bw: 0 Weight: 0
Link from switch 3 to switch 7 : ('96:ac:6f:e4:49:b4', 'ce:d9:94:48:16:12')  Delay: 0.01 Bw: 10.0 Weight: 5.
Link from host to switch 3 : ('fa:c4:2c:ea:4f:dd', '00:00:00:00:00:03')  Delay: 0 Bw: 0 Weight: 0
Link from switch 3 to switch 6 : ('52:62:c2:56:4e:0f', '92:39:90:e6:29:46')  Delay: 0.01 Bw: 10.0 Weight: 5.
Link from switch 6 to its own port 2 : ('92:39:90:e6:29:46', 6)  Delay: 0 Bw: 0 Weight: 0
Link from switch 4 to its own port 1 : (4, '8e:b5:77:d8:05:0b')  Delay: 0 Bw: 0 Weight: 0
Link from switch 4 to its own port 2 : (4, 'ee:f9:f6:81:0f:9e')  Delay: 0 Bw: 0 Weight: 0
Link from host to switch 4 : ('8e:b5:77:d8:05:0b', '00:00:00:00:00:04')  Delay: 0 Bw: 0 Weight: 0
Link from switch 5 to its own port 1 : (5, 'b6:7b:46:0c:78:9e')  Delay: 0 Bw: 0 Weight: 0
Link from host to switch 5 : ('b6:7b:46:0c:78:9e', '00:00:00:00:00:05')  Delay: 0 Bw: 0 Weight: 0
Link from switch 6 to its own port 1 : (6, 'c6:01:df:23:6f:27')  Delay: 0 Bw: 0 Weight: 0
Link from host to switch 6 : ('c6:01:df:23:6f:27', '00:00:00:00:00:06')  Delay: 0 Bw: 0 Weight: 0
Link from switch 7 to its own port 1 : (7, '6a:10:42:d3:ad:c8')  Delay: 0 Bw: 0 Weight: 0
Link from switch 7 to its own port 2 : (7, 'ce:d9:94:48:16:12')  Delay: 0 Bw: 0 Weight: 0
Link from host to switch 7 : ('6a:10:42:d3:ad:c8', '00:00:00:00:00:07')  Delay: 0 Bw: 0 Weight: 0

```

Figure 2.4 : Screenshot of Tree topology used

```
131 ****************************************
132 Shortest path from Host interface 00:00:00:00:00:04 to Host interface 00:00:00:00:00:06:
133
134  -> (Host Interface)00:00:00:00:00:04 -> (Switch Interface)8e:b5:77:d8:05:0b -> (Switch)4 -> (Switch Interface)
     ee:f9:f6:81:0f:9e -> (Switch Interface)9e:2f:c3:82:bd:15 -> (Switch)2 -> (Switch Interface)e6:13:7b:89:15:ae ->
     (Switch Interface)5e:b6:a6:e0:6f:46 -> (Switch)1 -> (Switch Interface)9a:2c:01:ae:52:85 -> (Switch Interface)
     5a:1b:70:10:d8:5d -> (Switch)3 -> (Switch Interface)52:62:c2:56:4e:0f -> (Switch Interface)92:39:90:e6:29:46 ->
     (Switch)6 -> (Switch Interface)c6:01:df:23:6f:27 -> (Host Interface)00:00:00:00:00:06
135
136 ****************************************
137 Shortest path from Host interface 00:00:00:00:00:04 to Host interface 00:00:00:00:00:07:
138
139  -> (Host Interface)00:00:00:00:00:04 -> (Switch Interface)8e:b5:77:d8:05:0b -> (Switch)4 -> (Switch Interface)
     ee:f9:f6:81:0f:9e -> (Switch Interface)9e:2f:c3:82:bd:15 -> (Switch)2 -> (Switch Interface)e6:13:7b:89:15:ae ->
     (Switch Interface)5e:b6:a6:e0:6f:46 -> (Switch)1 -> (Switch Interface)9a:2c:01:ae:52:85 -> (Switch Interface)
     5a:1b:70:10:d8:5d -> (Switch)3 -> (Switch Interface)96:ac:6f:e4:49:b4 -> (Switch Interface)ce:d9:94:48:16:12 ->
     (Switch)7 -> (Switch Interface)6a:10:42:d3:ad:c8 -> (Host Interface)00:00:00:00:00:07
140
141 ****************************************
142 Shortest path from Host interface 00:00:00:00:00:05 to Host interface 00:00:00:00:00:06:
143
144  -> (Host Interface)00:00:00:00:00:05 -> (Switch Interface)b6:7b:46:0c:78:9e -> (Switch)5 -> (Switch Interface)
     6a:2b:49:84:4f:56 -> (Switch Interface)0a:19:d9:77:bf:05 -> (Switch)2 -> (Switch Interface)e6:13:7b:89:15:ae ->
     (Switch Interface)5e:b6:a6:e0:6f:46 -> (Switch)1 -> (Switch Interface)9a:2c:01:ae:52:85 -> (Switch Interface)
     5a:1b:70:10:d8:5d -> (Switch)3 -> (Switch Interface)52:62:c2:56:4e:0f -> (Switch Interface)92:39:90:e6:29:46 ->
     (Switch)6 -> (Switch Interface)c6:01:df:23:6f:27 -> (Host Interface)00:00:00:00:00:06
145
146 ****************************************
147 Shortest path from Host interface 00:00:00:00:00:05 to Host interface 00:00:00:00:00:07:
148
149  -> (Host Interface)00:00:00:00:00:05 -> (Switch Interface)b6:7b:46:0c:78:9e -> (Switch)5 -> (Switch Interface)
     6a:2b:49:84:4f:56 -> (Switch Interface)0a:19:d9:77:bf:05 -> (Switch)2 -> (Switch Interface)e6:13:7b:89:15:ae ->
     (Switch Interface)5e:b6:a6:e0:6f:46 -> (Switch)1 -> (Switch Interface)9a:2c:01:ae:52:85 -> (Switch Interface)
     5a:1b:70:10:d8:5d -> (Switch)3 -> (Switch Interface)96:ac:6f:e4:49:b4 -> (Switch Interface)ce:d9:94:48:16:12 ->
     (Switch)7 -> (Switch Interface)6a:10:42:d3:ad:c8 -> (Host Interface)00:00:00:00:00:07
150
151 ****************************************
152 Shortest path from Host interface 00:00:00:00:00:06 to Host interface 00:00:00:00:00:07:
153
154  -> (Host Interface)00:00:00:00:00:06 -> (Switch Interface)c6:01:df:23:6f:27 -> (Switch)6 -> (Switch Interface)
     92:39:90:e6:29:46 -> (Switch Interface)52:62:c2:56:4e:0f -> (Switch)3 -> (Switch Interface)96:ac:6f:e4:49:b4 ->
     (Switch Interface)ce:d9:94:48:16:12 -> (Switch)7 -> (Switch Interface)6a:10:42:d3:ad:c8 -> (Host Interface)
     00:00:00:00:00:07
```

Figure 2.5 : Flows installed by SPR controller in Tree topology

Figures 2.3, 2.4, and 2.5 show the Tree topology diagram and its relevant log output. In Figure 2.4(Line 22), the link between Switch 1 and 2 is 50. However, due to Switch 1 acting as a bottleneck in this topology, there is no alternative route(with less combined weight) for traffic between Switch 4 and Switch 7. According to Figure 2.5(Line 137 & 139), traffic between Switch 4 and Switch 7 needs to go through the Switch 1 to Switch 2 link(with heavy weight). Topologies with such bottlenecks cannot provide better performance using Shortest Path routing.

3. Mesh Topology

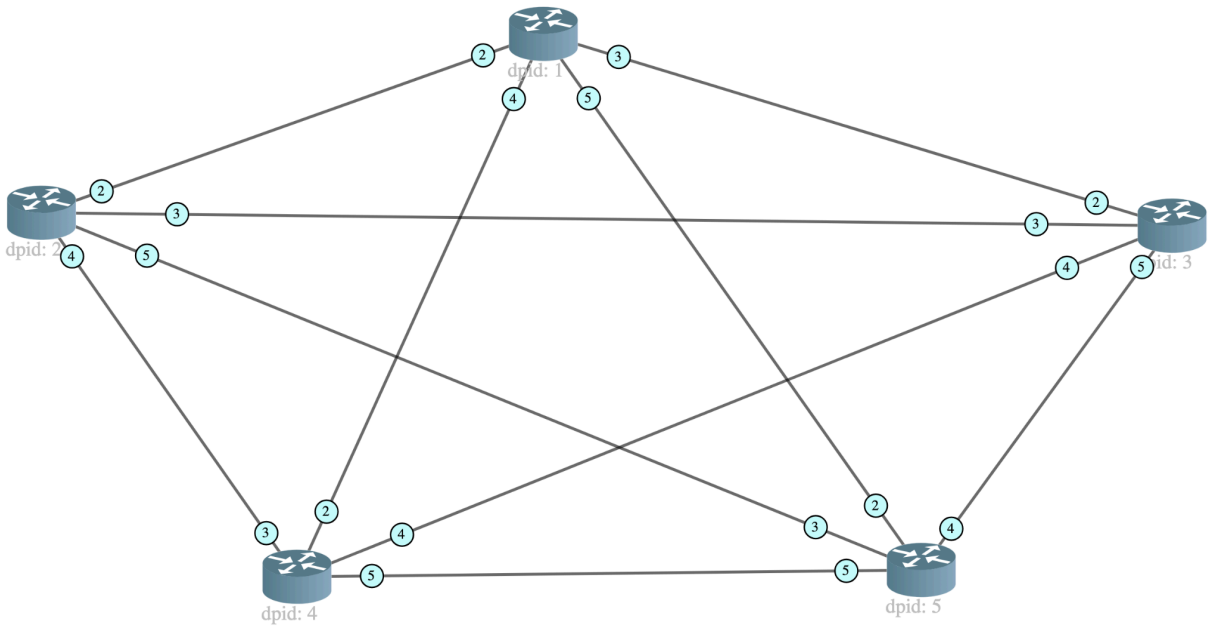


Figure 2.6: Mesh Topology with 5 switches, and 1 Host connected to each switch

```
Hosts:
Host 00:00:00:00:00:01
Host 00:00:00:00:00:02
Host 00:00:00:00:00:03
Host 00:00:00:00:00:04
Host 00:00:00:00:00:05
Switches:
Switch 1
Switch 2
Switch 3
Switch 4
Switch 5
Links:
Link from switch 1 to its own port 4 : (1, 'c2:f4:43:3d:99:56')  Delay: 0 Bw: 0 Weight: 0
Link from switch 1 to its own port 1 : (1, '1a:75:17:f5:9c:d8')  Delay: 0 Bw: 0 Weight: 0
Link from switch 1 to its own port 2 : (1, 'ae:22:39:2f:9a:cc')  Delay: 0 Bw: 0 Weight: 0
Link from switch 1 to its own port 5 : (1, '72:3d:9c:49:64:67')  Delay: 0 Bw: 0 Weight: 0
Link from switch 1 to its own port 3 : (1, '72:d6:7c:fc:85:b7')  Delay: 0 Bw: 0 Weight: 0
Link from switch 1 to switch 4 : ('c2:f4:43:3d:99:56', '9e:70:f0:f7:b9:c9')  Delay: 0.01 Bw: 1.0 Weight: 50.0
Link from host to switch 1 : ('1a:75:17:f5:9c:d8', '00:00:00:00:00:01')  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to switch 1 : ('ae:22:39:2f:9a:cc', '56:d9:46:cf:6e:77')  Delay: 0.01 Bw: 1.0 Weight: 50.0
Link from switch 1 to switch 5 : ('72:3d:9c:49:64:67', '3e:04:a9:cb:84:f5')  Delay: 0.01 Bw: 10.0 Weight: 5.0
Link from switch 1 to switch 3 : ('72:d6:7c:fc:85:b7', '16:bd:3b:fe:f5:9f')  Delay: 0.01 Bw: 1.0 Weight: 50.0
Link from switch 3 to its own port 2 : ('16:bd:3b:fe:f5:9f', 3)  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to its own port 2 : ('56:d9:46:cf:6e:77', 2)  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to its own port 1 : (2, '2a:98:5f:a5:58:33')  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to its own port 4 : (2, '1e:8c:32:0b:2b:e6')  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to its own port 5 : (2, '7a:40:05:0e:54:79')  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to its own port 3 : (2, 'b6:12:16:71:ed:86')  Delay: 0 Bw: 0 Weight: 0
Link from host to switch 2 : ('2a:98:5f:a5:58:33', '00:00:00:00:00:02')  Delay: 0 Bw: 0 Weight: 0
Link from switch 2 to switch 4 : ('1e:8c:32:0b:2b:e6', 'f6:86:4a:93:83:75')  Delay: 0.01 Bw: 10.0 Weight: 5.0
Link from switch 2 to switch 5 : ('7a:40:05:0e:54:79', '16:91:c1:08:c9:c2')  Delay: 0.01 Bw: 10.0 Weight: 5.0
Link from switch 2 to switch 3 : ('b6:12:16:71:ed:86', 'e6:87:bd:42:e4:f2')  Delay: 0.01 Bw: 10.0 Weight: 5.0
Link from switch 3 to its own port 4 : (3, '7a:15:0e:af:f7:b9')  Delay: 0 Bw: 0 Weight: 0
Link from switch 3 to its own port 1 : (3, '9e:3e:83:01:33:36')  Delay: 0 Bw: 0 Weight: 0
Link from switch 3 to its own port 5 : (3, '7e:38:2b:3c:e9:87')  Delay: 0 Bw: 0 Weight: 0
Link from switch 3 to its own port 3 : (3, 'e6:87:bd:42:e4:f2')  Delay: 0 Bw: 0 Weight: 0
Link from switch 3 to switch 4 : ('7a:15:0e:af:f7:b9', '66:ce:89:b8:fd:4c')  Delay: 0.01 Bw: 10.0 Weight: 5.0
Link from host to switch 3 : ('9e:3e:83:01:33:36', '00:00:00:00:00:03')  Delay: 0 Bw: 0 Weight: 0
Link from switch 3 to switch 5 : ('7e:38:2b:3c:e9:87', '9e:e8:70:eb:6f:9d')  Delay: 0.01 Bw: 10.0 Weight: 5.0
Link from switch 5 to its own port 2 : ('3e:04:a9:cb:84:f5', 5)  Delay: 0 Bw: 0 Weight: 0
Link from switch 4 to its own port 1 : (4, '1a:78:b4:23:78:ef')  Delay: 0 Bw: 0 Weight: 0
Link from switch 4 to its own port 4 : (4, '66:ce:89:b8:fd:4c')  Delay: 0 Bw: 0 Weight: 0
Link from switch 4 to its own port 2 : (4, '9e:70:f0:f7:b9:c9')  Delay: 0 Bw: 0 Weight: 0
Link from switch 4 to its own port 5 : (4, 'fe:d8:34:ba:27:3d')  Delay: 0 Bw: 0 Weight: 0
Link from switch 4 to its own port 3 : (4, 'f6:86:4a:93:83:75')  Delay: 0 Bw: 0 Weight: 0
Link from host to switch 4 : ('1a:78:b4:23:78:ef', '00:00:00:00:00:04')  Delay: 0 Bw: 0 Weight: 0
Link from switch 4 to switch 5 : ('fe:d8:34:ba:27:3d', 'ae:09:74:10:d6:3c')  Delay: 0.01 Bw: 10.0 Weight: 5.0
Link from switch 5 to its own port 5 : ('ae:09:74:10:d6:3c', 5)  Delay: 0 Bw: 0 Weight: 0
Link from switch 5 to its own port 4 : (5, '9e:e8:70:eb:6f:9d')  Delay: 0 Bw: 0 Weight: 0
Link from switch 5 to its own port 1 : (5, 'ee:b5:05:69:1c:7e')  Delay: 0 Bw: 0 Weight: 0
Link from switch 5 to its own port 3 : (5, '16:91:c1:08:c9:c2')  Delay: 0 Bw: 0 Weight: 0
Link from host to switch 5 : ('ee:b5:05:69:1c:7e', '00:00:00:00:00:05')  Delay: 0 Bw: 0 Weight: 0

```

Figure 2.7 : Screenshot of topology.txt log showing switches, host and links in Mesh topology



Figure 2.8 : Screenshot of shortest paths calculated for Mesh topology

Figures 2.6, 2.7 and 2.8 show the Mesh topology. In Figure 2.7(Line 19, 21 & 23), three links from Switch 1 to Switches 2, 3 and 4 have a weight of 50. The link between Switch 1 to Switch 5 has a weight of 5. According to Figure 2.8(Line 56-68), all traffic from Switch 1 to Switches 2, 3 and 4 is routed via Switch 5.

Part 3 : Congestion-aware Shortest Path Routing

1. Objective

The objective of Part 3 is to **modify the controller to dynamically route traffic based on current link utilization**, ensuring congestion-aware shortest paths.

2. Assumptions:

1. **Flow-level routing**: Routing decisions are made at the granularity of flows (identified by source-destination MAC pairs). Packet-level routing is not considered.

2. **Polling interval**: Link utilization is monitored every 5 seconds.

3. **Weight computation**: Link weight depends on:

    - **Bandwidth**: High-capacity links are preferred.

    - **Current utilization**: Links with higher utilization are penalized.

    - **Link delay**: Links with higher delays are penalized.

4. **Topology discovery**: Switches and links are detected dynamically using Ryu topology events

3 Implementation Overview

3.1 Controller Initialization

- The Ryu app initializes:

    - `datapaths`: Active switches.

    - `net`: NetworkX graph representing the topology.

    - `prev_link_load`: Stores previous TX bytes for each link.

    - `shortest_paths`: Stores the currently installed paths between nodes.

- A monitor thread is spawned to periodically request port statistics from switches.

- CSV logging is initialized for throughput, utilization, weight, delay, and bandwidth.

3.2 Topology Handling

- **Switch registration**: Each switch that joins the network is added as a node in the graph.
- **Link detection**: Each link is added as an edge with initial weight `1.0` and ports recorded.
- **Topology logging**: Current switches and links are printed for debugging.

3.3 Link Statistics

- Every 5 seconds, the controller requests port statistics from each switch.
- For each link:
    - `delta_tx` is computed as the difference in transmitted bytes since the last poll.
    - **Throughput (Mbps)** = `delta_tx * 8 / 1e6 / polling_interval`.
    - **Utilization** = `throughput / link bandwidth`.
    - **Link weight** = `α*(1/bandwidth) + β*delay + γ*utilization` (α, β, γ = 1.0).
- Bandwidth and delay are extracted using Linux `tc` commands for each interface.

3.4 Dynamic Shortest Path Computation

- Dijkstra's algorithm is run on the weighted network graph.
- If the shortest path for a source-destination pair changes, the new path is installed via flow rules.
- Only flows between known hosts are routed; unknown destinations are flooded.

3.5 Flow Installation

- Flow rules are installed with:
    - Priority: 100
    - Idle timeout: 10s
    - Hard timeout: 30s
- Actions forward packets along the calculated path.

3.6 CSV Logging

- The controller logs for every link at each polling interval:
    - `time`, `src`, `dst`, `throughput_mbps`, `utilization`, `weight`, `delay`, `bandwidth_mbps`.

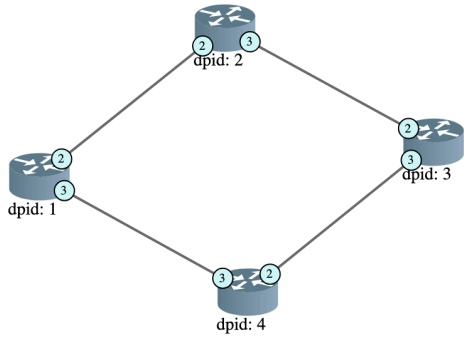
4 Testing:

Topology Tested:

1. Cyclic Topology
2. Tree Topology
3. Mesh Topology

4.1 Result Log:

Topology 1: Cyclic Topology




```
(py_3.9.6) baadalvm@baadalvm:~/ankita/mycopy/C/part3$ sudo python3.10 generate_topology2_congestion.py
*** Testing connectivity ***
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)

*** Congesting entire path 1 together ***

*** Congestion level: 2 streams per link ***
Running: h1 -> h2: iperf -c 10.0.0.2 -t 5 -P 2
Running: h2 -> h3: iperf -c 10.0.0.3 -t 5 -P 2
Baseline for h1->h2:
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.1 port 49854 connected with 10.0.0.2 port 5001
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-4.1330 sec   351 KBytes   695 Kbits/sec

Baseline for h2->h3:
------------------------------------------------------------
Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.2 port 40876 connected with 10.0.0.3 port 5001
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-4.5279 sec   896 KBytes  1.62 Mbits/sec


*** Congestion level: 5 streams per link ***
Running: h1 -> h2: iperf -c 10.0.0.2 -t 5 -P 5
Running: h2 -> h3: iperf -c 10.0.0.3 -t 5 -P 5
Baseline for h1->h2:
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.1 port 44134 connected with 10.0.0.2 port 5001
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-4.5720 sec   829 KBytes  1.48 Mbits/sec

Baseline for h2->h3:
------------------------------------------------------------
Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 85.3 KByte (default)
```

Log of Topology and Congestion generation script

```
Path changed 4->2: [4, 1, 2]
Path changed 4->3: [4, 3]
Link 1->2 | prev_tx: 0 | curr_tx: 3751538 | delta_bytes: 3751538 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 0.60 | weight: 0.71
Link 1->4 | prev_tx: 0 | curr_tx: 3593814 | delta_bytes: 3593814 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 0.58 | weight: 0.69
Link 3->4 | prev_tx: 0 | curr_tx: 3583218 | delta_bytes: 3583218 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 0.57 | weight: 0.68
Link 3->2 | prev_tx: 0 | curr_tx: 3583218 | delta_bytes: 3583218 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 0.57 | weight: 0.68
Path changed 1->3: [1, 4, 3]
Path changed 2->4: [2, 3, 4]
Path changed 3->1: [3, 4, 1]
Path changed 4->2: [4, 3, 2]
Link 4->3 | prev_tx: 0 | curr_tx: 3586684 | delta_bytes: 3586684 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 0.57 | weight: 0.68
Link 4->1 | prev_tx: 0 | curr_tx: 3743880 | delta_bytes: 3743880 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 0.60 | weight: 0.71
Link 1->2 | prev_tx: 3751538 | curr_tx: 9922464 | delta_bytes: 6170926 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 0.99 | weight: 1.10
Link 1->4 | prev_tx: 3593814 | curr_tx: 9754250 | delta_bytes: 6160436 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 0.99 | weight: 1.10
Link 4->3 | prev_tx: 3586684 | curr_tx: 9746380 | delta_bytes: 6159696 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 0.99 | weight: 1.10
Link 4->1 | prev_tx: 3743880 | curr_tx: 9914050 | delta_bytes: 6170170 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 0.99 | weight: 1.10
```

Log of RYU Controller script for updated weight and path changed Event

```
(py_3.9.6) baadalvm@baadalvm:~/ankita/mycopy/C/part3$ ryu-manager --observe-links ~/ankita/mycopy/C/part3/p4_ca_spr_level6.py --ofp-tcp-listen-port 7000
loading app /home/baadalvm/ankita/mycopy/C/part3/p4_ca_spr_level6.py
loading app ryu.topology.switches
loading app ryu.controller.ofp_handler
instantiating app /home/baadalvm/ankita/mycopy/C/part3/p4_ca_spr_level6.py of CongestionAwareRouting
Congestion-Aware Routing app initialized
instantiating app ryu.topology.switches of Switches
instantiating app ryu.controller.ofp_handler of OFPHandler
Datapath registered: 3
Datapath registered: 1
Switch 3 entered network.
Current Switches: [3, 1]
Current Links: []
Switch 1 entered network.
Current Switches: [3, 1]
Current Links: []
Datapath registered: 2
Datapath registered: 4
Switch 2 entered network.
Current Switches: [3, 1, 2, 4]
Current Links: []
Switch 4 entered network.
Current Switches: [3, 1, 2, 4]
Current Links: []
Link detected: 2:2 <-> 1:2
Current Switches: [3, 1, 2, 4]
Current Links: [(1, 2, {'port_src': 2, 'port_dst': 2, 'weight': 1.0})]
Link detected: 4:3 <-> 1:3
Current Switches: [3, 1, 2, 4]
Current Links: [(1, 2, {'port_src': 2, 'port_dst': 2, 'weight': 1.0}), (1, 4, {'port_src': 3, 'port_dst': 3, 'weight': 1.0})]
Link detected: 2:3 <-> 3:2
Current Switches: [3, 1, 2, 4]
Current Links: [(3, 2, {'port_src': 3, 'port_dst': 2, 'weight': 1.0}), (1, 2, {'port_src': 2, 'port_dst': 2, 'weight': 1.0}), (1, 4, {'port_src': 3, 'port_dst': 3, 'weight': 1.0})]
Link detected: 4:2 <-> 3:3
```

Log of RYU Controller detecting switches and links

- The network forms a **cyclic (ring) topology** with 4 switches (s1–s4) and 4 hosts (h1–h4), where each switch connects to its neighbor — s1–s2–s3–s4–s1 — forming a closed loop.

- Connectivity test (`pingAll`) shows **0% packet loss**, confirming full bidirectional reachability between all hosts.

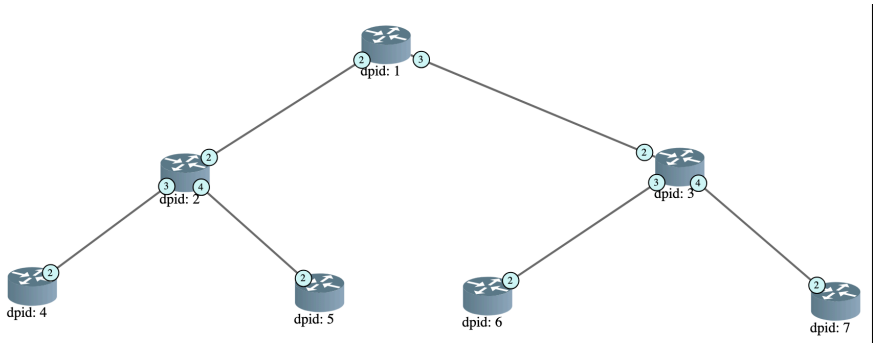
Congestion Generation and Measurement

- Controlled congestion was applied sequentially with **2, 5, and 8 parallel TCP streams per link** using `iperf`.
- The congested path primarily includes **h1→h2 (s1–s2)** and **h2→h3 (s2–s3)**.
- After each congestion phase, a baseline `iperf` measurement was run to assess link throughput.

Observed Bandwidth Trends

Congestion Level	h1→h2 Bandwidth	h2→h3 Bandwidth	Observation
2 streams	~0.695 Mbps	~1.62 Mbps	Initial load produces moderate throughput; h2→h3 slightly higher due to lower contention.
5 streams	~1.48 Mbps	~1.76 Mbps	Bandwidth increases as more parallel streams utilize available link capacity.
8 streams	~1.82 Mbps	~1.16 Mbps	Further increase in congestion causes slight instability — throughput no longer scales linearly.

Topology 2:Tree Topology:



Log of RYU Controller detecting switches

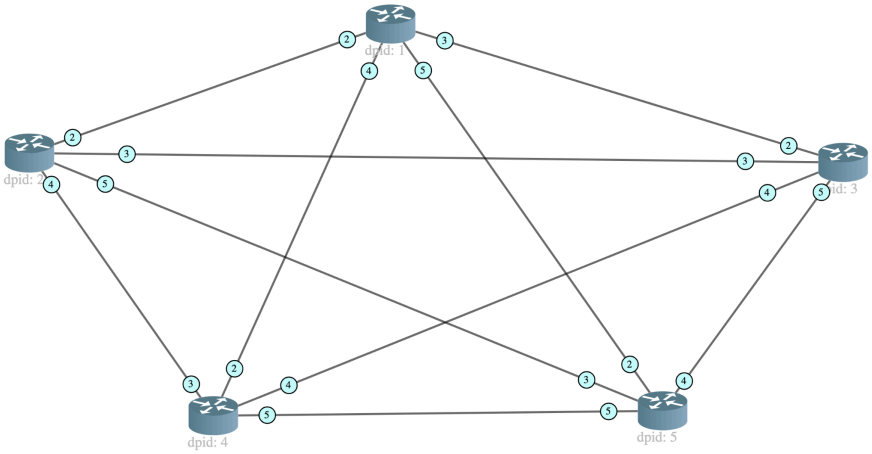
```
Current Switches: [6, 8, 4, 7, 3, 5, 2, 1]
Current Links: []
Link detected: 7:2 <-> 6:3
Current Switches: [6, 8, 4, 7, 3, 5, 2, 1]
Current Links: [(6, 7, {'port_src': 2, 'port_dst': 3, 'weight': 1.0})]
Link detected: 5:3 <-> 6:2
Current Switches: [6, 8, 4, 7, 3, 5, 2, 1]
Current Links: [(6, 7, {'port_src': 2, 'port_dst': 3, 'weight': 1.0}), (6, 5, {'port_src': 3, 'port_dst': 2, 'weight': 1.0})]
Link detected: 2:3 <-> 3:2
Current Switches: [6, 8, 4, 7, 3, 5, 2, 1]
```

Log of RYU Controller Link Detection

```
Link 2->1 | prev_tx: 15504316 | curr_tx: 16429110 | delta_bytes: 924794 | bandwidth: 1.0 Mbps | delay: 0.02s | utilization: 1.48 | weight: 2.50
Path changed 2->7: [2, 1, 5, 6, 7]
Path changed 7->2: [7, 6, 5, 1, 2]
Link 7->6 | prev_tx: 15662582 | curr_tx: 16607640 | delta_bytes: 945058 | bandwidth: 2.0 Mbps | delay: 0.01s | utilization: 0.76 | weight: 1.27
Link 7->8 | prev_tx: 15736714 | curr_tx: 16660222 | delta_bytes: 923508 | bandwidth: 2.0 Mbps | delay: 0.02s | utilization: 0.74 | weight: 1.26
Path changed 6->3: [6, 7, 8, 4, 3]
Path changed 3->6: [3, 4, 8, 7, 6]
```

Log of RYU Controller script for updated weight and path changed Event

Topology 3: Mesh Topology



```
(py_3.9.6) baadalvm@baadalvm:~/ankita/mycopy/C/part3$ ryu-manager --observe-links ~/ankita/mycopy/C/part3/p4_ca_spr_level6.py
loading app /home/baadalvm/ankita/mycopy/C/part3/p4_ca_spr_level6.py
loading app ryu.controller.ofp_handler
loading app ryu.topology.switches
loading app ryu.controller.ofp_handler
instantiating app /home/baadalvm/ankita/mycopy/C/part3/p4_ca_spr_level6.py of CongestionAwareRouting
Congestion-Aware Routing app initialized
instantiating app ryu.controller.ofp_handler of OFPHandler
instantiating app ryu.topology.switches of Switches
Datapath registered: 1
Switch 1 entered network.
Current Switches: [1]
Current Links: []
Datapath registered: 4
Datapath registered: 2
Datapath registered: 3
Datapath registered: 5
Switch 4 entered network.
Current Switches: [1, 4, 2, 3, 5]
Current Links: []
Switch 2 entered network.
Current Switches: [1, 4, 2, 3, 5]
Current Links: []
Switch 3 entered network.
Current Switches: [1, 4, 2, 3, 5]
Current Links: []
Switch 5 entered network.
Current Switches: [1, 4, 2, 3, 5]
Current Links: []
Link detected: 1:4 <-> 4:2
Current Switches: [1, 4, 2, 3, 5]
Current Links: [(1, 4, {'port_src': 4, 'port_dst': 2, 'weight': 1.0})]
Link detected: 1:3 <-> 3:2
Current Switches: [1, 4, 2, 3, 5]
Current Links: [(1, 4, {'port_src': 4, 'port_dst': 2, 'weight': 1.0}), (1, 3, {'port_src': 3, 'port_dst': 2, 'weight': 1.0})]
Link detected: 1:5 <-> 5:2
Current Switches: [1, 4, 2, 3, 5]
```

Log of RYU Controller detecting switches and links

```
Link 1->4 | prev_tx: 9592618 | curr_tx: 10217678 | delta_bytes: 625060 | bandwidth: 1.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 2.01
Link 1->3 | prev_tx: 9214286 | curr_tx: 9839446 | delta_bytes: 625160 | bandwidth: 1.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 2.01
Link 1->5 | prev_tx: 9214286 | curr_tx: 9839446 | delta_bytes: 625160 | bandwidth: 1.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 2.01
Link 1->2 | prev_tx: 9214286 | curr_tx: 9839446 | delta_bytes: 625160 | bandwidth: 1.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 2.01
Path changed 1->2: [1, 2]
Path changed 1->3: [1, 3]
Path changed 2->1: [2, 1]
Path changed 3->1: [3, 1]
Link 2->4 | prev_tx: 92762190 | curr_tx: 99012820 | delta_bytes: 6250630 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 1.11
Link 2->1 | prev_tx: 9891326 | curr_tx: 10516446 | delta_bytes: 625120 | bandwidth: 1.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 2.01
Link 2->3 | prev_tx: 92718208 | curr_tx: 98969298 | delta_bytes: 6251090 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 1.11
Link 2->5 | prev_tx: 92718208 | curr_tx: 98969298 | delta_bytes: 6251090 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 1.11
Link 3->5 | prev_tx: 93394690 | curr_tx: 99645030 | delta_bytes: 6250340 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 1.11
Link 3->4 | prev_tx: 93394690 | curr_tx: 99645030 | delta_bytes: 6250340 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 1.11
Link 3->1 | prev_tx: 10658886 | curr_tx: 11283876 | delta_bytes: 624990 | bandwidth: 1.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 2.01
Link 3->2 | prev_tx: 93488252 | curr_tx: 99738792 | delta_bytes: 6250540 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 1.11
Path changed 3->5: [3, 5]
Path changed 5->3: [5, 3]
Link 4->1 | prev_tx: 93826794 | curr_tx: 100078494 | delta_bytes: 6251700 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 1.11
Link 4->3 | prev_tx: 0 | curr_tx: 100078494 | delta_bytes: 100078494 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 16.01 | weight: 16.12
Link 4->2 | prev_tx: 93826794 | curr_tx: 100078494 | delta_bytes: 6251700 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 1.11
Link 4->5 | prev_tx: 93452572 | curr_tx: 99704372 | delta_bytes: 6251800 | bandwidth: 10.0 Mbps | delay: 0.01s | utilization: 1.00 | weight: 1.11
Path changed 4->3: [4, 5, 3]
Path changed 3->4: [3, 5, 4]
```

Log of RYU Controller script for updated weight and path changed Event

The log of Topology and Congestion Generation script confirms

- The topology was successfully created with 5 hosts and 5 switches.

- `pingAll` returned **0% packet loss (56/56 received)**, confirming correct link configuration and connectivity before congestion testing.

2. Topology Detection

- All 5switches (s1–s5) were successfully discovered and registered by the Ryu controller.

The controller dynamically detected 8 bidirectional links among the switches

- Each link was initialized with default weight = 1.0, bandwidth, and delay as defined by the topology script.
- The topology was thus fully connected and symmetric, enabling multiple alternative paths between any source–destination pair.

2. Dynamic Link Monitoring

- The controller periodically collected port statistics (bytes transmitted) and calculated:
    - Utilization = (Δbytes / bandwidth)
- Links with higher delay or utilization had higher weights, causing them to be less preferred during path computation.

3. Adaptive Path Computation

- After link discovery, the controller began computing paths for all switch pairs.
- show that shortest paths were installed dynamically in the switches' flow tables.

As link weights changed due to congestion or delay, the controller recomputed shortest paths:

4. Congestion Detection and Response

- The controller detected varying link utilization (0.04–0.08), indicating mild to moderate load levels.
- Links with higher delay (0.02s) or lower bandwidth (1 Mbps) quickly accumulated higher weights (~1.10), while others stayed at 0.55.
- Whenever link utilization increased, the controller routed flows through alternate, less congested paths.

Congestion Testing Summary

Congestion was introduced on the (s1-s2) , (s1,s3), (s1,s4) with increasing intensity — 5, 8, and 12 parallel `iperf` streams per link.

For each congestion level, baseline throughput measurements were taken immediately afterward to capture the residual link performance.

Congestion Level	Average Throughput (approx.)	Observation
5 streams/link	0.15 – 1.26 Mbit/s	Moderate congestion starts to appear, but all links remain active.
8 streams/link	0.13 – 1.33 Mbit/s	Bandwidth begins to fluctuate; lower values on early hops (h1–h2, h2–h3).
12 streams/link	0.11 – 0.69 Mbit/s	Heavy congestion evident; throughput sharply decreases on intermediate links.

References

1) https://ryu-sdn.org/
2) https://mininet.org/
3) https://osrg.github.io/ryu-book/en/html/traffic_monitor.html
4) https://ryu.readthedocs.io/en/latest/writing_ryu_app.html
5) https://networkx.org/